Training NeuralNector with Generative Adversarial Networks on Flowers 102

Issac Roy [1]

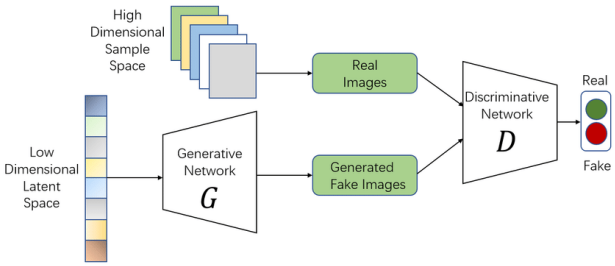

Figure 1. High-level GAN setup: the generator maps latent noise to images; the discriminator scores real vs. fake.

Abstract

This work trains DCGAN-style generative adversarial networks on MNIST and Oxford Flowers 102, progressing from a compact architecture for 28×28 digits to a deeper convolutional generator and discriminator for 64×64 RGB flower images. The resulting flower generator is deployed in NeuralNector, an interactive game in which human players act as the discriminator on grids of real versus synthetic images. Training is discussed in light of practical difficulties including discriminator dominance, vanishing gradients, and mode collapse, together with stabilizing modifications to the optimization procedure.

1. Introduction

Generative adversarial networks (GANs) (Goodfellow et al., 2014) are a framework for learning a model of a data distribution by pitting two neural networks against each other. A *generator* G maps samples z from a simple prior (e.g., Gaussian noise) to synthetic data $G(z)$ in the same space as real examples x. A *discriminator* D is a classifier trained to distinguish real data from generated samples. Training alternates between updating D to assign high probability to real x and low probability to $G(z)$, and updating G to produce samples that fool D. Figure 4 sketches the adversarial setup and layer structure used in this work.

The original minimax objective (Goodfellow et al., 2014) is

$$\min_G \max_D \; \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1-D(G(z)))]\,, \tag{1}$$

where $D(x) \in (0,1)$ is interpreted as the probability that x is real. In practice, G is often trained with a *non-saturating* loss that maximizes $\mathbb{E}_z[\log D(G(z))]$ instead of minimizing $\mathbb{E}_z[\log(1 - D(G(z)))]$, which can mitigate vanishing gradients when D rejects fakes with high confidence (Goodfellow et al., 2014). Our implementation follows the standard recipe of optimizing D and G with alternating stochastic gradient steps on losses equivalent to binary cross-entropy between D's output and labels for real vs. fake batches (with additional stabilizers described in Section 2). However, we found that the discriminator can become too accurate too quickly, which yields vanishing gradients for G and stalled learning (Arjovsky & Bottou, 2017; Salimans et al., 2016). Our solution is to skip the discriminator update if the discriminator's loss dips below a certain threshold, allowing G to catch up.

GAN training is notoriously fragile. As mentioned above, the discriminator can become too accurate too quickly, which yields vanishing gradients for G and stalled learning. *Mode collapse* occurs when G maps many latent vectors to a small set of outputs, limiting diversity (Arjovsky & Bottou, 2017; Salimans et al., 2016). Figure 2 shows an example of mode collapse on the Flowers 102 dataset where the generator only produces white images. Other pathologies include oscillations and sensitivity to hyperparameters (Arjovsky & Bottou, 2017). These issues motivated a large body of follow-on work (e.g., improved optimization and evaluation (Salimans et al., 2016; Arjovsky et al., 2017)); in this project we address practical instability with architectural scaling and training heuristics rather than attempting to formulate a new objective.

We began with the MNIST dataset and a compact convo-

[1]University of California, San Diego, La Jolla, USA. Correspondence to: Issac Roy <iroy@ucsd.edu, ucsd>.

Proceedings of the 40^{th} International Conference on Machine Learning, Honolulu, Hawaii, USA. PMLR 202, 2023.

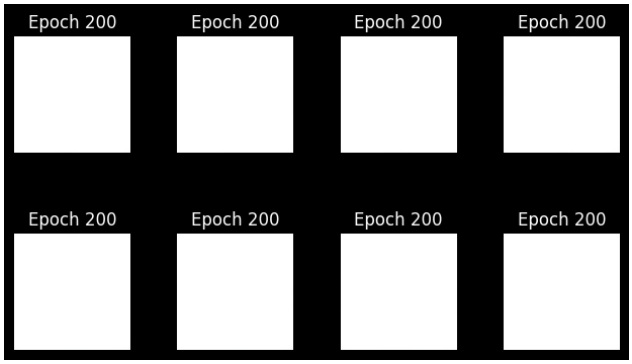

Figure 2. Mode collapse in the generator on the Flowers 102 dataset causes only white images to be generated.

lutional architecture in `GAN_MNIST.ipynb`: a shallow discriminator with convolutions, pooling, and fully connected layers, and a generator built from a linear projection to a small spatial grid followed by transposed convolutions to produce 28×28 grayscale digits. We then moved to the Oxford Flowers 102 dataset (Nilsback & Zisserman, 2008), which is substantially harder: natural color images at higher resolution require a deeper DCGAN-style design with more layers and 64×64 RGB outputs, developed in `GAN_pytorch.ipynb`. On Flowers we observed vanishing gradients, mode collapse, and a discriminator that reached near-optimality too early, suppressing learning in G. We mitigated these effects with practices such as label smoothing, asymmetric learning rates for G and D, and selective updates that skip or repeat discriminator steps when its loss indicates it is overpowering the generator; details appear in Section 2. The resulting generator powers NeuralNector, an interactive game in which human players act as discriminators on grids of real vs. GAN-generated flowers.

2. Methods

All models are implemented in PyTorch with the PyTorch Lightning training framework, using manual optimization so the generator and discriminator can be updated on different schedules with distinct learning rates.

2.1. Datasets and splits

MNIST. Grayscale digits are resized to 28×28. Inputs use the standard per-channel normalization to match the mean and variance. The training partition is split into approximately $55{,}000$ training and $5{,}000$ validation examples; the held-out test set is reserved for qualitative checks. This setting is used to validate the training loop and a lightweight convolutional architecture before scaling to natural images.

Figure 3. Samples from the generator at selected training epochs (flowers).

Oxford Flowers 102 (Nilsback & Zisserman, 2008). Images are resized and cropped to 64×64 RGB. Pixel values are mapped to $[-1, 1]$ via normalization with mean and standard deviation 0.5 per channel, which matches the `tanh` output range of the generator. The dataset provides only about $1{,}000$ images in each of the official training and validation splits; to increase per-epoch diversity and exposure to the class distribution, the training and validation splits are concatenated for optimization. The official test split isn't used during training and is kept for evaluation in Section 3. When training, optional data augmentation (random crops, flips, jitter, etc.) further increases variability for the flower domain.

2.2. Architectures: MNIST vs. Flowers

The same conceptual GAN objective (Equation (1)) is used throughout; the capacity and inductive bias of the networks are scaled with dataset difficulty. Figure 4 summarizes the layer stacks for both regimes, with G and D drawn side by side within each panel.

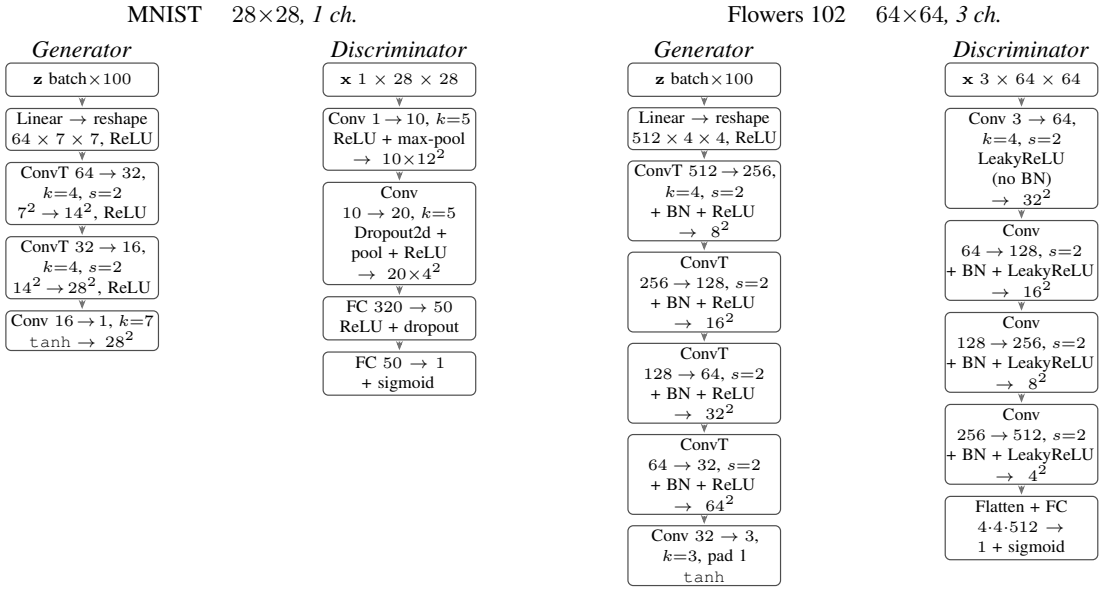

Figure 4. Neural architectures (`models.py`). *Left:* MNIST—G and D side by side; shallow D (pooling, dropout, no BN); G without BN. *Right:* Flowers—DCGAN-style D (BN except first conv, LeakyReLU) and G (transposed convs + BN + ReLU, `tanh`).

MNIST (28×28, 1 channel). The discriminator is a small CNN: two convolutional layers with 5×5 kernels, max pooling, dropout on the second conv block, and two fully connected layers ending in a single logit passed through a sigmoid. This shallow design is sufficient for aligned digits and trains quickly while limiting overfitting on a simple domain. The generator maps Gaussian noise of dimension $z \in \mathbb{R}^{100}$ through a linear layer to a $7 \times 7 \times 64$ tensor, applies two transposed convolutions to reach spatial size 28×28, and a final convolution with `tanh`. Batch normalization is omitted in this path to keep the starter model minimal and to avoid small-batch noise on early experiments; ReLU activations are used in the generator, and ReLU with pooling in the discriminator.

Flowers (64×64, 3 channels). Natural color flowers exhibit richer texture and spatial structure, so both networks follow a DCGAN-style deep convolutional layout with strided convolutions in D and transposed convolutions in G. The discriminator stacks four strided 4×4 convolutions with channel widths $64 \to 128 \to 256 \to 512$, batch normalization on all but the first layer (a common practice so the input statistics are not whitewashed), LeakyReLU (0.2) activations, and a linear layer to one output with sigmoid. BN and LeakyReLU help stabilize gradients when D must discriminate fine-grained natural detail; the first conv remains unnormalized so the network can learn low-level edge statistics directly from pixels. The generator projects $z \in \mathbb{R}^{100}$ to $4 \times 4 \times 512$, then upsamples through four transposed convolution stages ($512 \to 256 \to 128 \to 64 \to 32$ feature maps) each followed by batch normalization and ReLU, and a final 3×3 convolution to three channels with `tanh`. BN in G reduces internal covariate shift across upsampling stages and tends to improve training stability when many layers are stacked.

Convolutional, transposed-convolutional, and linear weights are initialized from $\mathcal{N}(0, 0.02^2)$ with zero bias initialization, following common GAN practice.

2.3. Loss and alternating updates

The discriminator is trained to minimize binary cross-entropy between its sigmoid output and soft targets for real vs. fake batches. Label smoothing with parameter 0.1 replaces hard labels 1 and 0 for real and fake with 0.9 and 0.1, which discourages D from becoming overconfident and mitigates vanishing gradients through the generator when fakes are trivially rejected (Salimans et al., 2016). The generator is trained to fool D, using BCE against targets of "real" (1) for generated samples (non-saturating form, consistent with Equation (1) and the discussion in the introduction).

2.4. Optimization and adaptive balancing

Both networks use Adam with betas $(0.5, 0.999)$. The base learning rate is 2×10^{-4} for the hyperparameter `lr`, but asymmetric effective rates are applied: the generator uses $\texttt{lr}_G = 2 \times \texttt{lr}$ (4×10^{-4}) and the discriminator $\texttt{lr}_D = \frac{1}{2} \times \texttt{lr}$ (1×10^{-4}). The intent is to slow D and accelerate G so the discriminator does not reach optimality too quickly—a regime in which G receives little useful gradient and training stalls.

Additional adaptive scheduling is implemented in the training step: if the averaged discriminator loss d_{loss} falls below

Table 1. Default hyperparameters for Flowers 102 training (unless varied in Section 3).

SETTING	VALUE
LATENT DIMENSION $\lvert\mathbf{z}\rvert$	100
BASE LR	2×10^{-4}
lr_G / lr_D	$2\times$ / $0.5\times$ BASE
LABEL SMOOTHING	0.1
ADAM (β_1, β_2)	$(0.5, 0.999)$
SKIP D STEP IF $d_{\text{loss}} <$	0.6
EXTRA G STEP IF $d_{\text{loss}} <$	0.4
FLOWERS BATCH SIZE	64
OUTPUT ACTIVATION	TANH ON G, SIGMOID ON D

0.6, the discriminator update for that step is skipped (D is already winning too easily; updating it further would harm G). If d_{loss} falls below 0.4, the generator is updated twice in one step (second draw of z) so G can recover when D is very strong. Together with label smoothing and asymmetric learning rates, this implements a simple heuristic schedule that pauses or emphasizes updates based on the relative strength of D and G, without changing the underlying minimax game.

Typical batch sizes are 128 for early MNIST experiments in `GAN_MNIST.ipynb` and 64 for Flowers in `GAN_pytorch.ipynb`, subject to GPU memory.

2.5. Selecting fakes for NeuralNector

After training, synthetic flowers for the web game are produced by sampling the generator and ranking candidates by the discriminator's output, interpreted as the model's estimated probability that an image is real. A large pool of candidates is generated (and optionally merged with previously generated images), all are scored under a fixed checkpoint, and the top 500 by discriminator score are written to the game's fake image archive. This heuristic filters out egregious failures and mode-collapsed samples that D confidently rejects, improving the quality of the human-facing "fake" set at the cost of biasing the pool toward samples the current D finds plausible. The same trained `gan_flowers_final.ckpt` checkpoint is loaded in production for inference.

3. Experiments

4. Conclusion

References

Arjovsky, M. and Bottou, L. Towards principled methods for training generative adversarial networks, 2017.

Arjovsky, M., Chintala, S., and Bottou, L. Wasserstein gan, 2017.

DeepBean. Understanding gans (generative adversarial networks). URL https://www.youtube.com/watch?v=RAa55G-oEuk.

Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y. Generative adversarial networks, 2014.

Isola, P., Zhu, J.-Y., Zhou, T., and Efros, A. A. Image-to-image translation with conditional adversarial networks, 2016.

Nerd, N. The math behind generative adversarial networks clearly explained! URL https://www.youtube.com/watch?v=Gib_kiXgnvA.

Nilsback, M. and Zisserman, A. Automated flower classification over a large number of classes. In *Proceedings of the Indian Conference on Computer Vision, Graphics and Image Processing*, 2008.

Outlier. Vq-gan — paper explanation. URL https://www.youtube.com/watch?v=wcqLFDXaDO8.

Salimans, T., Goodfellow, I., Zaremba, W., Cheung, V., Radford, A., and Chen, X. Improved techniques for training gans, 2016.

van den Oord, A., Vinyals, O., and Kavukcuoglu, K. Neural discrete representation learning, 2017.

Wang, X., Girshick, R., Gupta, A., and He, K. Non-local neural networks, 2017.